

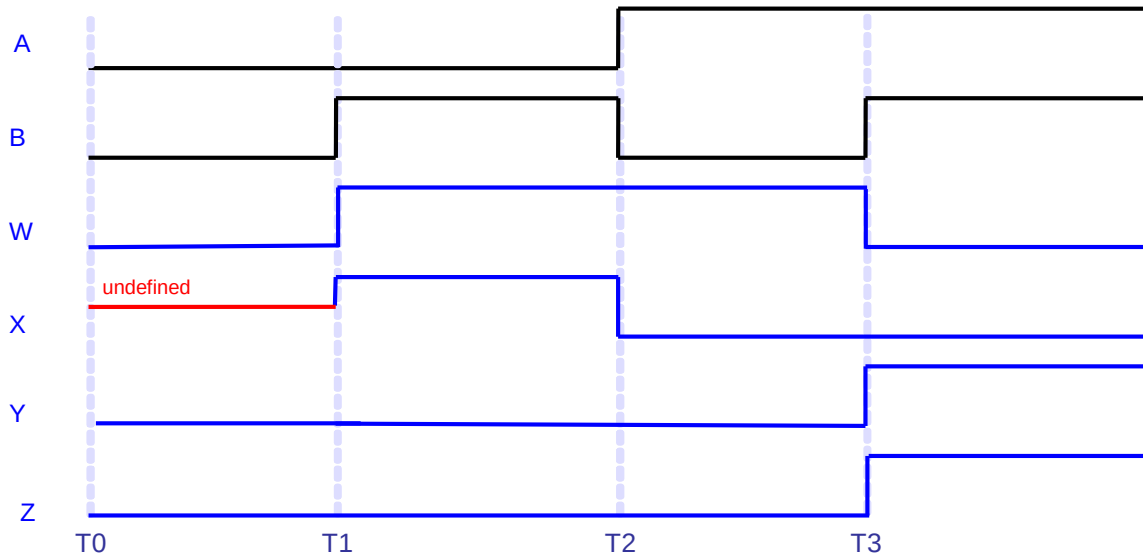
Tutorial 7 (SOLUTIONS)

NOTE THAT THERE ARE OTHER POSSIBLE SOLUTIONS TOO...

SS1

The always block is only triggered/executed when there is an event on B.

At T0,T1,T2,T3: X changes based on "old value" of W while Z follows "new value" of Y.



At t=T0	A	B	W	X	Y	Z	Deferred
Initial Condition	X	X	X	X	X	X	
B changes to 0	0	0	X	X	X	X	
$W \leq A \wedge B$;	0	0	X	X	X	X	$W \leq 0$
$X \leq \sim W$;	0	0	X	X	X	X	
$Y = A \& B$;	0	0	X	X	0	X	
$Z \leq Y$;	0	0	X	X	0	X	$Z \leq 0$
Assignment			0	X	0	0	

At t=T1	A	B	W	X	Y	Z	Deferred
Initial Condition	0	0	0	X	0	0	
B changes	0	1	0	X	0	0	
$W \leq A \wedge B$;	0	1	0	X	0	0	$W \leq 1$
$X \leq \sim W$;	0	1	0	1	0	0	
$Y = A \& B$;	0	1	0	1	0	0	
$Z \leq Y$;	0	1	0	1	0	0	$Z \leq 0$
Assignment			1	1	0	0	

At t=T2	A	B	W	X	Y	Z	Deferred
Initial Condition	0	1	1	1	0	0	
B changes	1	0	1	1	0	0	
$W \leq A \wedge B$;	1	0	1	1	0	0	$W \leq 1$
$X \leq \sim W$;	1	0	1	0	0	0	
$Y = A \& B$;	1	0	1	0	0	0	
$Z \leq Y$;	1	0	1	0	0	0	$Z \leq 0$
Assignment			1	0	0	0	

At t=T3	A	B	W	X	Y	Z	Deferred
Initial Condition	1	0	1	0	0	0	
B changes	1	1	1	0	0	0	
$W \leq A \wedge B$;	1	1	1	0	0	0	$W \leq 0$
$X \leq \sim W$;	1	1	1	0	0	0	
$Y = A \& B$;	1	1	1	0	1	0	
$Z \leq Y$;	1	1	1	0	1	0	$Z \leq 1$
Assignment			0	0	1	1	

*X represents an unknown/undefined value.

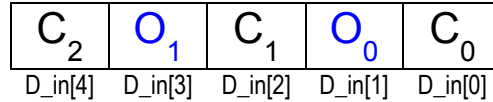
Question 1

```

module Dev_Encrypt (input [4:0] D_in, output reg [2:0] D_out);

always @ (D_in) //work out another solution in dataflow style!!
begin
    if ( D_in[3] == 0 && D_in[1] == 0 )
        D_out <= {D_in[4], D_in[2], D_in[0] };
    else if ( D_in[3] == 0 && D_in[1] == 1 )
        D_out <= {D_in[0], D_in[4], D_in[2] };
    else if ( D_in[3] == 1 && D_in[1] == 0 )
        D_out <= {D_in[2], D_in[0], D_in[4] };
    else
        D_out <= { (~D_in[4]) , (~D_in[2]), (~D_in[0]) };
end
endmodule

```



Question 2

// Solution 1 – Behavioral Style of Modeling

```

module customized_mux1 ( input [3:0] A, B, C, D, E,
                        input [2:0] S, output reg [3:0] T );
reg [3:0] tmp;

always @ ( S, A, B, C, D, E )
begin
    case (S)
        3'b000 : tmp = A;
        3'b001 : tmp = B;
        3'b010 : tmp = A;
        3'b011 : tmp = C;
        3'b100 : tmp = A;
        3'b101 : tmp = D;
        3'b110 : tmp = A;
        default : tmp = E;
    endcase
    T <= tmp;
end
endmodule

```

// Solution 2 – Dataflow Style of Modeling

```

module customized_mux2 ( input [3:0] A, B, C, D, E,
                        input [2:0] S, output [3:0] T);
assign T = S[2] ?
    ( S[1] ? ( S[0] ? E : A ) : ( S[0] ? D : A ) )
    : ( S[1] ? ( S[0] ? C : A ) : ( S[0] ? B : A ) );

endmodule

```

Question 3

Behavioral Style of Modeling (Using if statements to do the adding is another way to solve this!)

```

module mul_beh ( input [2:0] x, y, output reg [5:0] z);

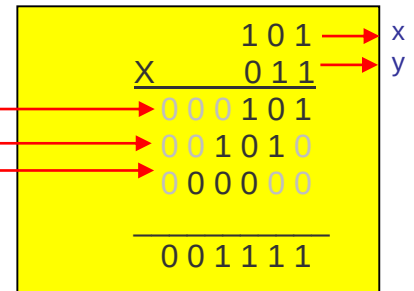
reg [5:0] w1, w2, w3, temp;

always @ ( x , y )
begin
    w1 = {3'b000, x[2], x[1],x[0]};
    w2 = {2'b00, x[2], x[1], x[0], 1'b0}; //w1*2
    w3 = {1'b0, x[2], x[1], x[0], 2'b00}; //w1*4

    case (y)
        3'b000 : temp = 6'b000000;
        3'b001 : temp= w1;
        3'b010 : temp= w2;
        3'b011 : temp= w2+w1;
        3'b100 : temp= w3;
        3'b101 : temp= w3+w1;
        3'b110 : temp= w3+w2;
        default : temp= w3+w2+w1;
    endcase
    z<=temp;
end

endmodule

```



Dataflow Style of Modeling

```

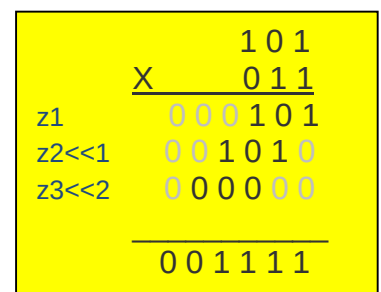
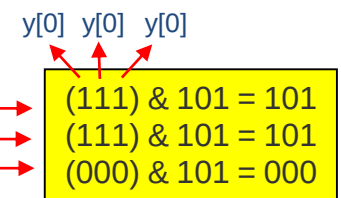
module mul_d (input [2:0] x, y, output [5:0] z);

wire [5:0] z1, z2, z3;

//Replication Operator! {n { }}
assign z1 = {3{y[0]}} & x;
assign z2 = {3{y[1]}} & x;
assign z3 = {3{y[2]}} & x;
assign z = z1 + (z2<<1) + (z3<<2);

endmodule

```



Question 4

```
module jkff ( input D, CLK, CLR, S, output reg Q, output QB);  
  
always @ ( negedge CLK, negedge CLR )  
  
begin  
    if ( CLR == 0 )  
        Q <= 0;  
    else  
        begin  
            if ( S == 0 )  
                Q <= 1;  
            else  
                Q <= D;  
        end  
end  
  
    assign QB = ~Q;  
  
endmodule
```